

# P1123R0

## Editorial Guidance for merging P0019r8 and P0528r3

Published Proposal, 8 June 2018

**This version:**

<https://github.com/ORNL/cpp-proposals-pub/blob/master/P1123/P1123r0.html>

**Author:**

[Daniel Sunderland](#)

**Audience:**

LWG

**Project:**

ISO JTC1/SC22/WG21: Programming Language C++

---

## Abstract

Merge proposed changes in P0019r8 and P0528r3

## Table of Contents

### 1 Proposal

#### § 1. Proposal

Text in blockquotes is not proposed wording.

If P0019r8 and P0528r3 are both accepted, apply the following changes to the compare-exchange wording in [atomics.ref.operations]

```
bool compare_exchange_weak(T& expected, T desired, memory_order success, memory_order failure) const noexcept;
```

```
bool compare_exchange_strong(T& expected, T desired, memory_order success, memory_order failure) const noexcept;
```

```
bool compare_exchange_weak(T& expected, T desired, memory_order order = memory_order_seq_cst) const noexcept;
```

```
bool compare_exchange_strong(T& expected, T desired, memory_order order = memory_order_seq_cst) const noexcept;
```

*Effects:* Retrieves the value in `expected`. It then atomically compares the value representation of the value referenced by `*ptr` for equality with that previously retrieved from `expected`, and if `true`, replaces the value referenced by `*ptr` with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `success`, and if the comparison is `false`, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`. If and only if the comparison is `false` then, after the atomic operation, the ~~contents of the memory~~ value in `expected` ~~are~~ is replaced by the value read from the value referenced by `*ptr` during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations [intro.races] on the value referenced by `*ptr`. Otherwise, these operations are atomic load operations on that memory.